Formalizing α-equivalence equations

Flávio L. C. de Moura[1], **Luís A. M. Sales**[1]

[1]Departamento de Ciência da Computação
Universidade de Brasília – Brasília, DF – Brazil

flaviomoura@unb.br, sales.luis@aluno.unb.br

Abstract. *Syntax with binding constructors is pervasive in Computer Science and Mathematics, particularly in the formal specification and implementation of logics and programming languages. Characterizing terms that differ only by the names of their bound variables, i.e., terms in the same α-equivalence class, is of great importance. A number of different definitions of α-equivalence can be found in the current literature, employing various approaches, such as nameless representations (e.g., de Bruijn indices) or explicit-name approaches based on substitution/renaming or swapping. In this paper, we prove the equivalence between several definitions of α-equivalence in the context of named variables using the Rocq proof assistant.*

1. Introduction

This work studies α-equivalence, a fundamental problem in both Computer Science and Mathematics that concerns the characterization of terms differing only in the names of their bound variables. Several approaches tackle this problem, such as de Bruijn notation [de Bruijn 1972], in which α-equivalent terms have a unique representation; the locally nameless representation [Charguéraud 2011], in which bound variables are represented by de Bruijn indices and free variables by named variables; and the nominal approach [Gabbay and Pitts 2002], in which variables are named; α-equivalence must be handled explicitly and renaming is done by swapping variable names. In the context of the untyped λ-calculus with named variables, we present four different notions of α-equivalence and prove that they are mutually equivalent in the Rocq proof assistant[1] [The Rocq Development Team 2025].

The contributions of this work are as follows:

1. Four alternative definitions of α-equivalence, compared and related: we present four distinct inductive definitions of α-equivalence for the pure λ-calculus, each capturing a different design choice:
   (a) *aeq* (Notation: $=_a$): the classical definition in the nominal context;
   (b) *aeqp* (Notation: $=_p$): a Pitts-style definition based on freshness of variables;
   (c) *aeqs* (Notation: $=_s$): a congruence generated by a single axiom based on swapping;
   (d) *Alpha* (Notation: $=_\alpha$): an equivalence defined by a dual context relation on variables and terms [Apinis and Ahman 2026].
2. A machine-checked proof that $=_a$, $=_p$, $=_s$ and $=_\alpha$ are mutually equivalent.

[1]Version 9.1.1

This work is developed with the Rocq proof assistant, and this article is generated directly from a Rocq source file. Consequently, the formal statements and proof scripts shown in this article are the actual Rocq code, and every claim labeled as a lemma or a theorem has been machine-verified. The source code is freely available at

`https://github.com/flaviodemoura/aeq-equations`

In compliance with SBC guidelines, the authors declare that generative AI tools were used as assistants to improve the spelling, grammar, and phrasing of the text and, in minor parts, of the proof scripts. All conceptual design decisions, theoretical formalizations, proofs, and the final text have been reviewed, validated, and approved by the authors, who take full responsibility for the integrity and accuracy of the content as a whole.

2. Preliminaries

The language of the untyped λ-calculus as presented in standard textbooks [Barendregt 1984, Barendregt and Manzonetto 2022] is given by

$$t ::= x \mid \lambda_x.t \mid t\ t \tag{1}$$

Variables are represented by atoms, which are structureless entities with a decidable equality: distinct names denote distinct atoms, and hence distinct variables. The nominal approach to abstract syntax was introduced by Gabbay and Pitts [Gabbay and Pitts 2002], motivated by the need for a mathematically rigorous and uniform treatment of variable binding and α-equivalence. The central observation of their work is that the standard set-theoretic treatment of syntax with binders, which typically requires working with equivalence classes of terms under α-renaming, can be replaced by a concrete, first-order presentation in which names are treated as primitive objects equipped with a notion of *freshness*. Rather than quotienting by α-equivalence, terms are considered as concrete syntactic objects and equivalence is captured by the action of finite permutations of names. This approach yields clean inductive characterizations of α-equivalence that are amenable to mechanical verification. The nominal approach has since been widely adopted in proof assistants and formalization frameworks, most notably in Nominal Isabelle [Urban and Kaliszyk 2012], and has been applied to the formalization of type theories, process calculi, and programming language semantics. The present work follows the nominal tradition and employs the Metalib library [Weirich, S. et al.] to support the development in Rocq.

In a more recent work, Apinis and Ahman [Apinis and Ahman 2026] continue following the above framework of untyped λ-calculus. Their main observation on the topic of α-equivalence is that "we do not need to consider the full context of the respective subexpression, but only the respective abstraction variables outside the subexpression." (p. 3-4); thus they develop an inductively defined structure to account for the context of both variables and terms. Both the nominal and the dual context approaches deal with variables without resorting to purely nameless representations such as de Bruijn indices. However, the dual context approach builds extra structure on top of the standard λ-calculus, with the advantage that the inference rules require no fresh-name generation machinery, nor the swap operation from the nominal approach; instead, the correspon-

dence between bound variables is resolved through their relative positions within both contexts.

3. Definitions and properties of α-equivalence

The first definition, which can be found in [Urban et al. 2004, Lima and de Moura 2023, Ayala-Rincón et al. 2020, Domínguez and Fernández 2021], is given by the following rules:

Definition 1.

$$\frac{t_1 =_a t_1' \qquad t_2 =_a t_2'}{t_1\ t_2 =_a t_1'\ t_2'}\ (aeq_app) \qquad \frac{t_1 =_a t_2}{\lambda_x.t_1 =_a \lambda_x.t_2}\ (aeq_abs_same)$$

$$(aeq_var)\ \frac{}{x =_a x} \qquad \frac{x \neq y \qquad x \notin fv(t_2) \qquad t_1 =_a (y\ x)t_2}{\lambda_x.t_1 =_a \lambda_y.t_2}\ (aeq_abs_diff)$$

where $(y\ x)t_2$ denotes the term obtained from t_2 by swapping the names x and y. The building block of the nominal approach is the *swap* operation, a transposition of two names, from which arbitrary finite permutations can be built. The swap operation is first defined on atoms (left) and then extended to terms (right):

$$(x\ y)z := \begin{cases} y & \text{if } z = x \\ x & \text{if } z = y \\ z & \text{otherwise} \end{cases} \qquad (x\ y)t := \begin{cases} (x\ y)z & \text{if } t \text{ is a variable } z \\ \lambda_{(x\ y)z}.(x\ y)t_1 & \text{if } t = \lambda_z.t_1 \\ (x\ y)t_1\ (x\ y)t_2 & \text{if } t = t_1\ t_2 \end{cases}$$

The second definition, which can be found in the seminal paper [Gabbay and Pitts 2002], is given by the following rules:

Definition 2.

$$(p_var)\ \frac{}{x =_p x} \qquad \frac{t_1 =_p t_1' \qquad t_2 =_p t_2'}{t_1\ t_2 =_p t_1'\ t_2'}\ (p_app)$$

$$\frac{z \notin \{x, y\} \cup fv(t_1) \cup fv(t_2) \qquad (x\ z)t_1 =_p (y\ z)t_2}{\lambda_x.t_1 =_p \lambda_y.t_2}\ (p_abs)$$

and it differs from Definition 1 only in the abstraction case, which is characterized by a single rule, where the α-equivalence of $\lambda_x.t_1$ and $\lambda_y.t_2$ is established through a more symmetric use of swapping on both terms. Rather than renaming only one side, one takes a fresh variable z (distinct from both x and y and not occurring free in either t_1 or t_2), and all occurrences of x in t_1 (respectively y in t_2) are transposed with z. The resulting terms are then required to be $=_p$-equivalent. This symmetric treatment avoids the asymmetry present in Definition 1 and simplifies certain aspects of the metatheory. The third definition, unlike the previous ones, takes the form of a congruence closure: it is defined as the smallest congruence relation containing a single swap-based renaming axiom. It can be seen as a nominal version of Church's definition [Church 1941]:

Definition 3.

$$\frac{}{t =_s t}\ (s_refl) \qquad \frac{t_1 =_s t_2}{t_2 =_s t_1}\ (s_sym) \qquad \frac{t_1 =_s t_2 \qquad t_2 =_s t_3}{t_1 =_s t_3}\ (s_trans)$$

$$\frac{t_1 =_s t_2}{\lambda_x.t_1 =_s \lambda_x.t_2}\ (s_abs_same) \qquad \frac{x \neq y \qquad y \notin fv(t)}{\lambda_y.(x\ y)t =_s \lambda_x.t}\ (s_abs_diff)$$

$$\frac{t_1 =_s t_1' \qquad t_2 =_s t_2'}{t_1\ t_2 =_s t_1'\ t_2'}\ (s_app)$$

The fourth definition, written $=_\alpha$, is taken from [Apinis and Ahman 2026]. It is built in two levels, one for variables containing three rules (*GlobalVA*, *HereVA* and *ThereVA*), and one for terms (*VarA* for equivalence on variables, *AppA* for function application and *AbsA* for abstraction). Bearing in mind that the authors use inverted lists (*xs*::*x*, where *x* is the head and *xs* the tail of the list), *VarA* states that if a variable context relates two variables under α-equivalence, then the corresponding variable terms are α-equivalent, while *AppA* keeps the variable context of matching α-equivalences and simply applies the corresponding subterms in a new application term. *AbsA* captures the relation between bound variables in term abstractions and the variable context. *GlobalVA* states that "in case of empty contexts, variable must be related to itself." (p. 4), while, for *HereVA*, "if neither context is empty, the head values of each context are related." (idem), and, for *ThereVA*, "variables a and b at lower positions in the context can be related only if neither a nor b appears above in their respective context." (idem).

Definition 4.

$$\frac{}{\langle [];x\rangle =_{v\alpha} \langle [];x\rangle}\ (GlobalVA) \qquad \frac{}{\langle xs :: x;x\rangle =_{v\alpha} \langle ys :: y;y\rangle}\ (HereVA)$$

$$\frac{\langle xs;a\rangle =_{v\alpha} \langle ys;b\rangle \qquad x \neq a \qquad y \neq b}{\langle xs :: x;a\rangle =_{v\alpha} \langle ys :: y;b\rangle}\ (ThereVA)$$

$$\frac{\langle xs;x\rangle =_{v\alpha} \langle ys;y\rangle}{\langle xs;x\rangle =_{\alpha} \langle ys;y\rangle}\ (VarA) \qquad \frac{\langle xs;f\rangle =_{\alpha} \langle ys;g\rangle \qquad \langle xs;t\rangle =_{\alpha} \langle ys;u\rangle}{\langle xs;f\ t\rangle =_{\alpha} \langle ys;g\ u\rangle}\ (AppA)$$

$$\frac{\langle xs :: x;t\rangle =_{\alpha} \langle ys :: y;u\rangle}{\langle xs;\lambda x.\ t\rangle =_{\alpha} \langle ys;\lambda y.\ u\rangle}\ (AbsA)$$

3.1. Properties

The next theorem reflects the fact that (Definition 1), i.e., =*a* is stable under swapping, which is used to prove that =*a* is an equivalence relation:

```
Theorem aeq_swap: ∀ t1 t2 x y, t1 =a t2 ↔ (swap x y t1) =a (swap x y t2).
```

As for $=a$, we need the stability of $=p$ to conclude that it is an equivalence relation.

```
Theorem aeqp_swap: ∀ t1 t2 x y, t1 =p t2 ↔ (swap x y t1) =p (swap x y t2).
```

The next lemma is a key property that relates the dual context with the nominal approach. It shows how dual contexts must be related so that an arbitrary swapping preserves $=_\alpha$.

```
Lemma alpha_rename_gen: ∀ t x y s l, (∀ z, z \in fv t → ⟪s; vswap y x z⟫ =vα ⟪l; z⟫) → ⟪s; swap y x t⟫ =α ⟪l; t⟫.
```

Proof. The proof is by induction on t. The interesting case is when $t = \lambda_z.t_1$ that is proved as follows:

$$\dfrac{\dfrac{\dfrac{\dfrac{\langle\!\langle s :: (y\ x)z; (y\ x)w\rangle\!\rangle =v\alpha \langle\!\langle l :: z; w\rangle\!\rangle}{w \in fv(t_1) \to \langle\!\langle s :: (y\ x)z; (y\ x)w\rangle\!\rangle =v\alpha \langle\!\langle l :: z; w\rangle\!\rangle}(\to_{\mathrm{i}})}{\forall z, z \in fv(t_1) \to \langle\!\langle s :: (y\ x)z; (y\ x)z\rangle\!\rangle =v\alpha \langle\!\langle l :: z; z\rangle\!\rangle}(\forall_i)}{\langle\!\langle s :: (y\ x)z; (y\ x)t_1\rangle\!\rangle =\alpha \langle\!\langle l :: z; t_1\rangle\!\rangle}(\text{I.H.})}{\langle\!\langle s; (y\ x)\lambda_z.t_1\rangle\!\rangle =\alpha \langle\!\langle l; \lambda_z.t_1\rangle\!\rangle}\ AbsA$$

where $\langle\!\langle s :: (y\ x)z; (y\ x)w\rangle\!\rangle =v\alpha \langle\!\langle l :: z; w\rangle\!\rangle$ is proved by considering all possible cases between the variables x, y, z and w, and the hypothesis $(\forall z, z \in fv(t) \to \langle\!\langle s; (y\ x)z\rangle\!\rangle = v\alpha \langle\!\langle l; z\rangle\!\rangle)$. □

The previous lemma is the key result to prove the following lemma, which has the same structure as rule (*s_abs_diff*) of Definition 3:

```
Lemma alpha_rename: ∀ t x y, x ≠ y → x \notin fv t → Abs x (swap y x t) =α Abs y t.
```

Proof. Note that *alpha_rename* shares the same goal as *alpha_rename_gen*, but it has a different set of assumptions. Rather than dealing with a third variable within both contexts, it relates only two. Since we use *alpha_rename_gen* to prove it, it is unsurprising that this third variable appears in the proof. However, only the case they are all different is worthy of note, since the other case simply follows from the definition. The following proof tree solves this case,

$$\dfrac{\dfrac{\dfrac{\dfrac{\dfrac{}{\langle[]; z\rangle =_{v\alpha} \langle[]; z\rangle}(GlobalVA) \quad \dfrac{\dfrac{}{z \in fv(t)}(H_{\mathrm{fv}'}) \quad \dfrac{}{x \notin fv(t)}(H_{\mathrm{fv}})}{x \neq z} \quad \dfrac{}{y \neq z}(H_{z\neq y})}{\langle[x]; z\rangle =_{v\alpha} \langle[y]; z\rangle}(ThereVA)}{\langle[x]; (y\ x)z\rangle =_{v\alpha} \langle[y]; z\rangle}(\text{DEF. vswap})}{\langle[x]; (y\ x)t\rangle =_{\alpha} \langle[y]; t\rangle}(alpha_rename_gen)}{\lambda_x.(y\ x)t =_{\alpha} \lambda_y.t}(AbsA)$$

As shown in [Apinis and Ahman 2026], variables can be removed from the context, as long as they are shadowed by earlier ones with the same value. In the next lemma, we prove a stronger result showing that if the contexts have the same length and their last variable is the same, then this common variable can be removed and α-equivalence is preserved:

```
Lemma alpha_drop_same_gen : ∀ t1 t2 s l x, length s = length l → ⟨⟨ [x] ++ s; t1⟩⟩ =α
⟨⟨ [x] ++ l; t2⟩⟩ → ⟨⟨s; t1⟩⟩ =α ⟨⟨l; t2⟩⟩.
```

As a corollary, named *alpha_drop_same*, we take s and l as empty lists, which is a key step in proving the next lemma,

```
Lemma alpha_push_swap : ∀ x y t1 t2, ⟨⟨ [x]; t1⟩⟩ =α ⟨⟨ [y]; t2⟩⟩ →
                                     ⟨⟨ []; t1⟩⟩ =α ⟨⟨ []; (swap y x t2) ⟩⟩.
```

Note that *alpha_push_swap* directly relates how one may perform a swap operation and preserve an α-equivalence relation. We first proceed by using *alpha_rename_gen* to establish that *t2* under the original context is α-equivalent to itself swapped under the *t1* variable context. Then, by transitivity, we show that *t2* swapped is α-equivalent to *t1* under the same variable context, and *alpha_drop_same* allows us to drop the variable in its context to achieve *alpha_push_swap*. In the next section, we prove that =*a*, =*p*, =*s* and =α are mutually equivalent.

4. Relating the definitions of α-equivalence

We first prove that Definition 1 implies Definition 2:

```
Theorem aeq_aeqp: ∀ t1 t2, t1 =a t2 → t1 =p t2.
```

Proof. The proof is by induction on the derivation of *t1* =*a t2*. The only non-trivial case is that of the rule *aeq_abs_diff*, where the fresh variable z introduced by the rule must be eliminated in favor of the binding variables x and y using the rule *p_abs*. Suppose that *t1* =*a t2* was generated by the rule *aeq_abs_diff* then both *t1* and *t2* are abstractions, say *t1* = $\lambda_x.t_1'$ and *t2* = $\lambda_y.t_2'$. Our goal is to prove that $\lambda_x.t_1' =_p \lambda_y.t_2'$ and the induction hypothesis is $(x\ z)t_1' =_p (y\ z)t_2'$ for a fresh variable z. The complete proof is given by the following derivation tree:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{t_1' =_p (y\ x)t_2'}\,(\text{I.H.})}{(x\ z)t_1' =_p (x\ z)(y\ x)t_2'}\,(aeqp_swap) \qquad \dfrac{}{(x\ z)(y\ x)t_2' =_p (y\ z)t_2'}\,(aeqp_swap_swap)}{(x\ z)t_1' =_p (y\ z)t_2'}\,(aeqp_trans)}{\lambda_x.t_1' =_p \lambda_y.t_2'}\,(p_abs)$$

□ We now prove that Definition 2 implies Definition 3:

```
Theorem aeqp_aeqs: ∀ t1 t2, t1 =p t2 → t1 =s t2.
```

Proof. By induction on the derivation of *t1* =*p t2*. Again, the only non-trivial case is that of the rule *p_abs*, where the fresh variable z introduced by the rule must be eliminated in favor of the binding variables x and y using the rule *s_abs_diff*. If *t1* =*p t2* was generated by the rule *p_abs* then both t_1 and t_2 are abstractions, say *t1* = $\lambda_x.t_1'$ and *t2* = $\lambda_y.t_2'$. Our goal is to prove that $\lambda_x.t_1' =_s \lambda_y.t_2'$ and the induction hypothesis is $(x\ z)t_1' =_s (y\ z)t_2'$ for

a fresh variable z. The complete proof is given by the following derivation tree:

$$\dfrac{(\star)\ \dfrac{}{\lambda_x.t_1' =_s \lambda_z.(x\ z)t_1'} \qquad (\Delta)\ \dfrac{(\text{I.H.})\ \dfrac{}{(x\ z)t_1' =_s (y\ z)t_2'}}{\lambda_z.(x\ z)t_1' =_s \lambda_z.(y\ z)t_2'} \qquad \dfrac{\dfrac{}{\lambda_y.t_2' =_s \lambda_z.(y\ z)t_2'\ (\star)}}{\lambda_z.(y\ z)t_2' =_s \lambda_y.t_2'\ (\diamond)}}{\lambda_z.(x\ z)t_1' =_s \lambda_y.t_2'\ (\nabla)}}{\lambda_x.t_1' =_s \lambda_y.t_2'\ (\nabla)}$$

where (∇) is the rule *s_trans*, (Δ) is *s_abs_same*, $(\diamond)$ is *s_sym* and $(\star)$ is *s_abs_diff*. □ The next lemma shows that Definition 3 implies Definition 4. Using the proofs in [Apinis and Ahman 2026] that $=\alpha$ is reflexive, symmetric, and transitive, and considering that the proof strategy is by induction on *t1* $=s$ *t2*, all cases except the abstraction are trivial. For the abstraction with the same abstraction variables, we are done by lemma *AlphaConvAbs* from [Apinis and Ahman 2026], and for abstractions with different variables we close the proof with lemma *alpha_rename*.

`Lemma` aeqs_to_Alpha: $\forall$ *t1 t2*, *t1* $=$`s` *t2* $\rightarrow$ *t1* $=\alpha$ *t2*.

Proof. The variable and application cases map directly to the constructors. For abstractions with the same variables, the goal follows by theorem *AlphaConvAbs* that corresponds to the rule (*aeqs_abs_same*) of Definition 3. For abstractions with distinct variables, we are done with an application of *alpha_rename*. □

In order to close the dependency graph between all four definitions, it remains to prove that $=_\alpha$ implies $=a$:

`Lemma` Alpha_to_aeq: $\forall$ *t1 t2*, *t1* $=\alpha$ *t2* $\rightarrow$ *t1* $=$`a` *t2*.

Proof. We proceed by structural induction on *t1*. The interesting case is abstraction, whose goal is given by $\lambda_x.t_1 =a\ t_2$. In this case, t_2 must be an abstraction, say $t_2 = \lambda_y.t_3$, and we have 2 subcases:

1. If $x = y$ then we are done by (I.H.) followed by lemma *alpha_drop_same*.

2. If $x \neq y$ $(H_{\neq})$ then we have the following derivation:

$$\dfrac{(H_{\neq})\ \dfrac{}{x \neq y} \qquad \dfrac{(\text{H})\ \dfrac{}{\lambda_x.t_1 =_\alpha \lambda_y.t_3} \qquad (H_{\text{fv}})\ \dfrac{}{x \notin fv(\lambda_x.t_1)}\ (\star)}{x \notin fv(t_3)} \qquad \dfrac{\dfrac{}{\lambda_x.t_1 =_\alpha \lambda_y.t_3}\ (\text{H})}{t_1 =_\alpha (y\ x)t_3}\ (\diamond) \qquad \dfrac{}{t_1 =_a (y\ x)t_3}\ (\text{I.H.})}{\lambda_x.t_1 =_a \lambda_y.t_3}\ (\Delta)$$

and we are done with the (I.H.), *alpha_fv_equiv* $(\star)$, *alpha_push_swap* $(\diamond)$ and *aeq_abs_diff* (Δ). □

5. Conclusion and Future Work

We presented a Rocq formalization in which four inductive definitions of α-equivalence for the pure λ-calculus with named variables are proved to coincide: the classical nomi-

nal definition $=_a$, its symmetric Pitts-style variant $=_p$, the congruence closure $=_s$ generated by a single swap-based renaming axiom, and the dual context relation $=_\alpha$ of [Apinis and Ahman 2026].

For future work, we plan to extend the formalization to syntax extended with metavariables, which play a central role in unification algorithms. In nominal unification [Urban et al. 2004], metavariables are subject to freshness constraints, and the interaction between binding, renaming, and the constraints imposed on metavariables is precisely what makes the α-equivalence theory of such syntax nontrivial.

References

Apinis, K. and Ahman, D. (2026). A simple formalization of alpha-equivalence. arXiv:2507.10181 [cs.LO]. Accessed: 2026-04-14. https://doi.org/10.48550/arXiv.2507.10181.

Ayala-Rincón, M., Fernández, M., and Nantes-Sobrinho, D. (2020). On Nominal Syntax and Permutation Fixed Points. *Logical Methods in Computer Science*, Vol 16, Issue 1.

Barendregt, H. and Manzonetto, G. (2022). *A Lambda Calculus Satellite*. College Publications.

Barendregt, H. P. (1984). *The Lambda Calculus: Its Syntax and Semantics*. Number v. 103 in Studies in Logic and the Foundations of Mathematics. North-Holland; Elsevier Science Pub. Co, Amsterdam ; New York : New York, N.Y, rev. ed edition.

Charguéraud, A. (2011). The Locally Nameless Representation. *Journal of Automated Reasoning*, pages 1–46.

Church, A. (1941). *The Calculi of Lambda-Conversion*, volume 6 of *Annals of Mathematics Studies*. Princeton University Press, Princeton, NJ.

de Bruijn, N. G. (1972). Lambda Calculus Notation With Nameless Dummies, a Tool for Automatic Formula Manipulation, With Application To the Church-Rosser Theorem. *Indagationes Mathematicae (Proceedings)*, 75(5):381–392.

Domínguez, J. and Fernández, M. (2021). Nominal syntax with atom substitutions. *Journal of Computer and System Sciences*, 119:34–59.

Gabbay, M. J. and Pitts, A. M. (2002). A New Approach to Abstract Syntax with Variable Binding. *Formal Aspects of Computing*, 13(3-5):341–363.

Lima, M. J. D. and de Moura, F. L. C. (2023). A Formalized Extension of the Substitution Lemma in Coq. *EPTCS*, 389:80–95.

The Rocq Development Team (2025). The Rocq Prover. https://doi.org/10.5281/zenodo.15149629.

Urban, C. and Kaliszyk, C. (2012). General Bindings and Alpha-Equivalence in Nominal Isabelle. *Logical Methods in Computer Science*, Volume 8, Issue 2.

Urban, C., Pitts, A. M., and Gabbay, M. J. (2004). Nominal Unification. *Theoretical Computer Science*, 323(1-3):473–497.

Weirich, S. et al. Coq Metalib package. https://github.com/plclub/metalib.